

Linux Kiosk Environment Deployment & Remote Access Project

Author: Chontele Coleman

Date: 02/05/2026

Project Overview

This project focused on building and configuring a lightweight Linux kiosk-style environment designed for controlled usage and remote administration. The goal was to simulate a real-world IT or security scenario where a system runs a restricted interface while still allowing secure remote troubleshooting through SSH.

Rather than only focusing on commands, this project emphasized **practical troubleshooting, structured configuration management, and secure remote diagnostics**.

Objectives

- Create a dedicated project workspace within the Linux home directory.
- Configure kiosk-related files and scripts using terminal-based editing.
- Establish secure remote access via SSH.
- Practice diagnostics and troubleshooting after remote login.
- Validate proper file placement, permissions, and execution.

Environment & Tools Used

- Ubuntu Linux
- SSH Remote Access
- Nano Text Editor
- Terminal CLI
- Linux File Permissions & Directory Management

Project Setup

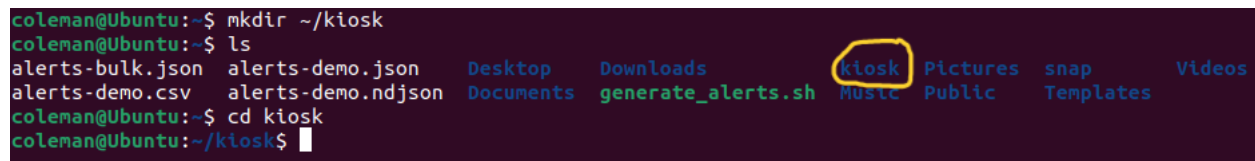
The project workspace was organized inside a dedicated directory to keep files structured and isolated from the rest of the system.

Example structure:

`~/kiosk`

By placing configuration files inside the directory/folder helped maintain a clean and clear environment and mirrored real world practices for deployment organization.

Figure 1: Directory Listing

A terminal window screenshot with a dark purple background. The first command is 'mkdir ~/kiosk'. The second command is 'ls', which lists files and directories: 'alerts-bulk.json', 'alerts-demo.json', 'Desktop', 'Downloads', 'kiosk', 'Pictures', 'snap', and 'Videos'. The third command is 'cd kiosk', and the fourth is '~/.kiosk\$'. The 'kiosk' directory is highlighted with a yellow circle.

```
coleman@Ubuntu:~$ mkdir ~/kiosk
coleman@Ubuntu:~$ ls
alerts-bulk.json  alerts-demo.json  Desktop  Downloads  kiosk  Pictures  snap  Videos
alerts-demo.csv  alerts-demo.ndjson Documents  generate_alerts.sh  Music  Public  Templates
coleman@Ubuntu:~$ cd kiosk
coleman@Ubuntu:~/.kiosk$
```

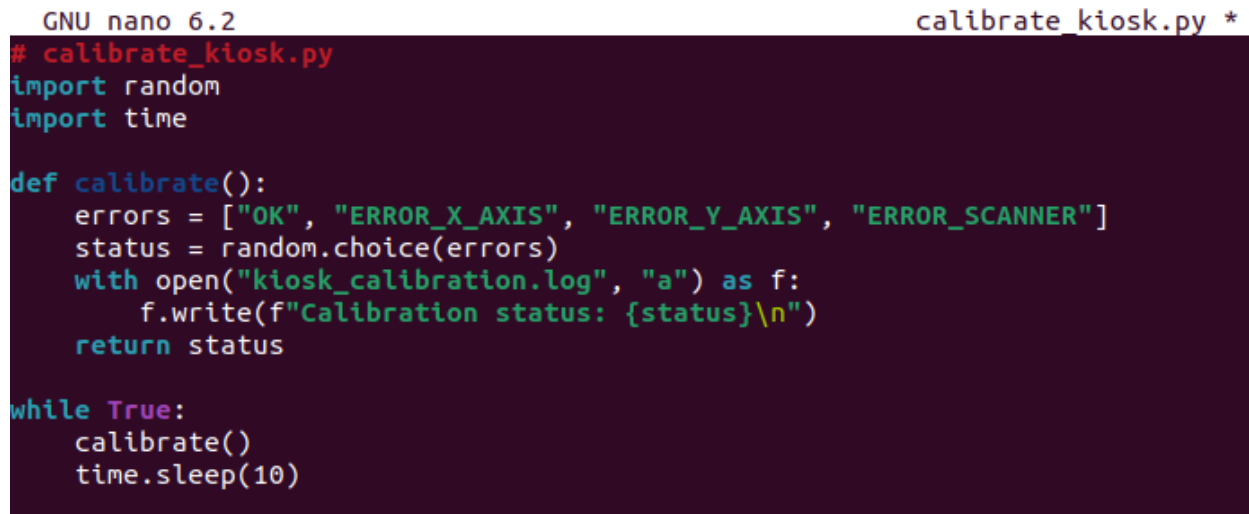
Configuration Process

Configuration files were edited directly from the terminal using Nano. This approach reinforces practical Linux administration skills since many production systems rely heavily on CLI-based management rather than graphical tools.

Key actions included:

- Creating/editing configuration files
- Saving secure settings
- Ensuring proper syntax and structure

Figure 2: Python script editing via nano command

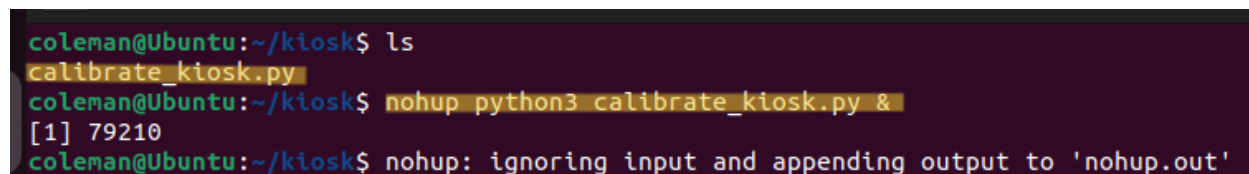


```
GNU nano 6.2                                     calibrate_kiosk.py *
# calibrate_kiosk.py
import random
import time

def calibrate():
    errors = ["OK", "ERROR_X_AXIS", "ERROR_Y_AXIS", "ERROR_SCANNER"]
    status = random.choice(errors)
    with open("kiosk_calibration.log", "a") as f:
        f.write(f"Calibration status: {status}\n")
    return status

while True:
    calibrate()
    time.sleep(10)
```

Figure 3: Script running in the background command



```
colemant@Ubuntu:~/kiosk$ ls
calibrate_kiosk.py
colemant@Ubuntu:~/kiosk$ nohup python3 calibrate_kiosk.py &
[1] 79210
colemant@Ubuntu:~/kiosk$ nohup: ignoring input and appending output to 'nohup.out'
```

Environment Setup & Failure Simulation

To begin, I created simulated kiosk log files to replicate hardware behavior and operational alerts.

Mock Scanner & Cutter Logs

The scanner and cutter system was simulated through a log file containing both normal operation messages and a failure condition.

Commands used:

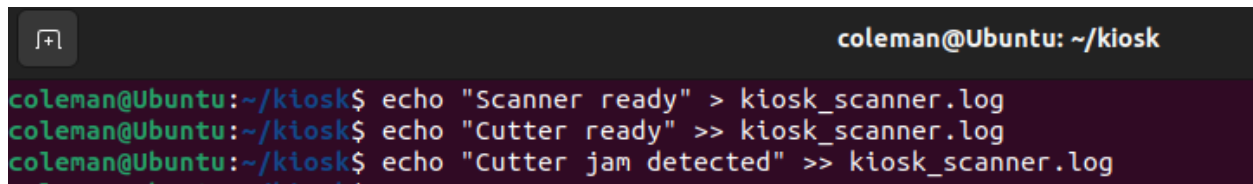
```
echo "Scanner ready" > kiosk_scanner.log
```

```
echo "Cutter ready" >> kiosk_scanner.log
```

```
echo "Cutter jam detected" >> kiosk_scanner.log
```

This allowed me to practice parsing logs for operational faults and to identify failure patterns.

Figure 4: Log creation and contents

A terminal window titled 'coleman@Ubuntu: ~/kiosk' showing three commands being executed: 'echo "Scanner ready" > kiosk_scanner.log', 'echo "Cutter ready" >> kiosk_scanner.log', and 'echo "Cutter jam detected" >> kiosk_scanner.log'.

```
coleman@Ubuntu:~/kiosk$ echo "Scanner ready" > kiosk_scanner.log
coleman@Ubuntu:~/kiosk$ echo "Cutter ready" >> kiosk_scanner.log
coleman@Ubuntu:~/kiosk$ echo "Cutter jam detected" >> kiosk_scanner.log
```

Simulated Kiosk Database with Permission Failure

A mock database file was created to replicate a common support issue where services fail due to incorrect permissions.

```
echo "key1=12345" > kiosk_db.txt
```

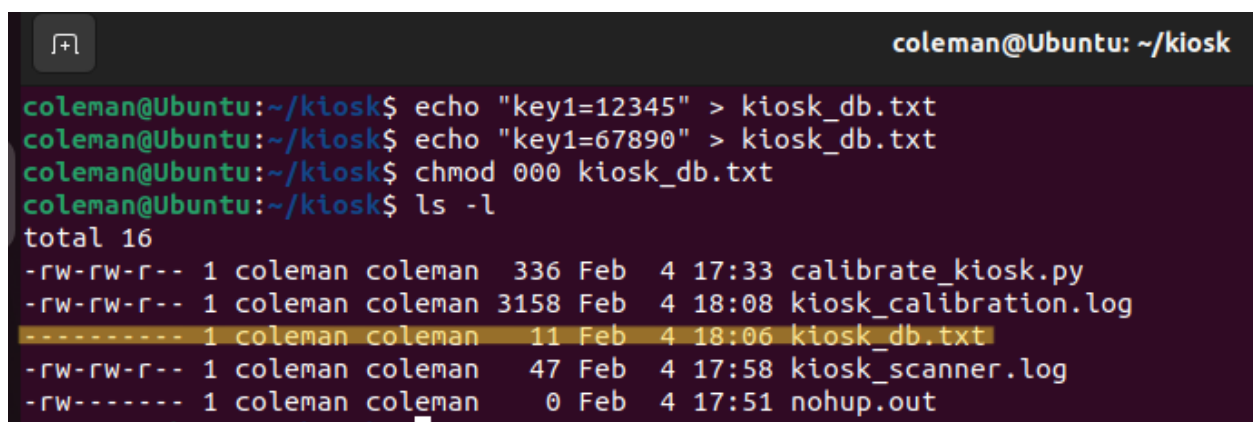
```
echo "key2=67890" >> kiosk_db.txt
```

```
chmod 000 kiosk_db.txt
```

Setting permissions to 000 intentionally prevented access to the file, forcing a permissions-based troubleshooting workflow.

This simulated real-world incidents where services cannot read configuration files due to improper ownership or access levels.

Figure 5: Permissions changed and ls -l output

A terminal window titled 'coleman@Ubuntu: ~/kiosk' showing four commands: 'echo "key1=12345" > kiosk_db.txt', 'echo "key1=67890" > kiosk_db.txt', 'chmod 000 kiosk_db.txt', and 'ls -l'. The output of 'ls -l' is shown, with the line for 'kiosk_db.txt' highlighted in yellow, showing permissions '-rw-rw-r--' and owner 'coleman'.

```
coleman@Ubuntu:~/kiosk$ echo "key1=12345" > kiosk_db.txt
coleman@Ubuntu:~/kiosk$ echo "key1=67890" > kiosk_db.txt
coleman@Ubuntu:~/kiosk$ chmod 000 kiosk_db.txt
coleman@Ubuntu:~/kiosk$ ls -l
total 16
-rw-rw-r-- 1 coleman coleman 336 Feb  4 17:33 calibrate_kiosk.py
-rw-rw-r-- 1 coleman coleman 3158 Feb  4 18:08 kiosk_calibration.log
-rw-rw-r-- 1 coleman coleman 11 Feb  4 18:06 kiosk_db.txt
-rw-rw-r-- 1 coleman coleman 47 Feb  4 17:58 kiosk_scanner.log
-rw-rw-r-- 1 coleman coleman 0 Feb  4 17:51 nohup.out
```

Introducing Realistic System Issues

To make the environment behave like a malfunctioning production kiosk, several problems were deliberately introduced.

Calibration Failures

Errors were logged into:

```
kiosk_calibration.log
```

This simulated devices failing alignment checks or startup calibration routines.

Scanner/Cutter Malfunction

The “Cutter jam detected” entry represented a mechanical failure scenario requiring log inspection.

Service Failure Simulation

The calibration process was intentionally stopped:

```
pkill -f calibrate_kiosk.py
```

This mimicked a crashed or stalled service.

Network Failure Simulation

Two separate network issues were introduced:

Block outbound web traffic:

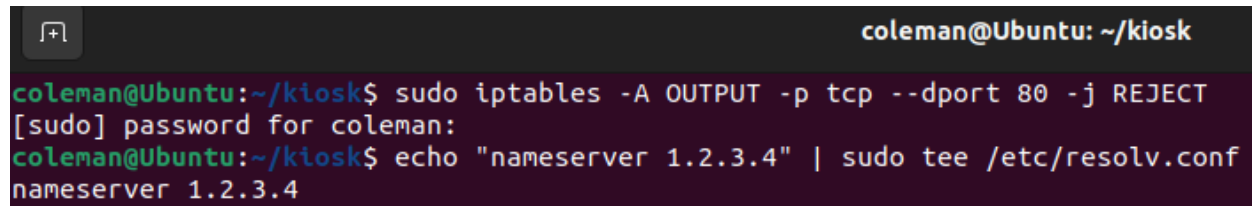
```
sudo iptables -A OUTPUT -p tcp --dport 80 -j REJECT
```

Break DNS resolution:

```
echo "nameserver 1.2.3.4" | sudo tee /etc/resolv.conf
```

These changes caused tools like `curl` to fail, reproducing real connectivity troubleshooting scenarios.

Figure 6: iptables rule and resolv.conf

A terminal window with a dark background. The prompt is 'coleman@Ubuntu: ~/kiosk'. The user enters 'sudo iptables -A OUTPUT -p tcp --dport 80 -j REJECT'. The prompt changes to '[sudo] password for coleman:'. The user then enters 'echo "nameserver 1.2.3.4" | sudo tee /etc/resolv.conf'. The output is 'nameserver 1.2.3.4'.

```
coleman@Ubuntu: ~/kiosk
coleman@Ubuntu:~/kiosk$ sudo iptables -A OUTPUT -p tcp --dport 80 -j REJECT
[sudo] password for coleman:
coleman@Ubuntu:~/kiosk$ echo "nameserver 1.2.3.4" | sudo tee /etc/resolv.conf
nameserver 1.2.3.4
```

Diagnostics & Investigation Phase

After creating failures, I moved into a structured diagnostic workflow using standard Linux tools.

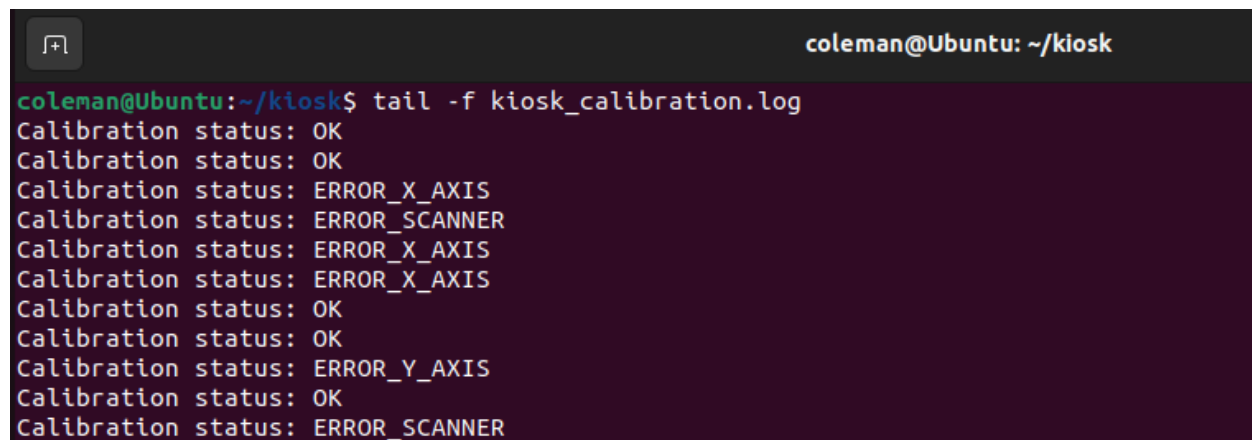
Reviewing Calibration Logs

```
tail -f ~/kiosk/kiosk_calibration.log
```

```
grep ERROR ~/kiosk/kiosk_calibration.log
```

This allowed continuous monitoring and targeted searching for failure messages. Also to focus on recurring error patterns and confirm whether the calibration script was running.

Figure 7 & 8: Tail output and grep results

A terminal window with a dark background. The prompt is 'coleman@Ubuntu: ~/kiosk'. The user enters 'tail -f kiosk_calibration.log'. The output shows a series of 'Calibration status' messages: 'OK', 'OK', 'ERROR_X_AXIS', 'ERROR_SCANNER', 'ERROR_X_AXIS', 'ERROR_X_AXIS', 'OK', 'OK', 'ERROR_Y_AXIS', 'OK', and 'ERROR_SCANNER'.

```
coleman@Ubuntu: ~/kiosk
coleman@Ubuntu:~/kiosk$ tail -f kiosk_calibration.log
Calibration status: OK
Calibration status: OK
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_SCANNER
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_X_AXIS
Calibration status: OK
Calibration status: OK
Calibration status: ERROR_Y_AXIS
Calibration status: OK
Calibration status: ERROR_SCANNER
```

```
colem@Ubuntu: ~/kiosk
colem@Ubuntu:~/kiosk$ grep ERROR kiosk_calibration.log
Calibration status: ERROR_SCANNER
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_SCANNER
Calibration status: ERROR_SCANNER
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_Y_AXIS
Calibration status: ERROR_SCANNER
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_X_AXIS
Calibration status: ERROR_SCANNER
Calibration status: ERROR_X_AXIS
```

File Permission Diagnostics

```
ls -l ~/kiosk/kiosk_db.txt
```

The output confirmed that permissions were set to `-----`, meaning no read/write access existed.

Fix applied:

```
chmod 644 kiosk_db.txt
```

Running this command restores proper accessibility for services.

Figure 9: Correct permissions applied

```
colem@Ubuntu: ~/kiosk
colem@Ubuntu:~/kiosk$ ls -l kiosk_db.txt
----- 1 colem colem 11 Feb 4 18:06 kiosk_db.txt
colem@Ubuntu:~/kiosk$ chmod 644 kiosk_db.txt
colem@Ubuntu:~/kiosk$ ls -l
total 20
-rw-rw-r-- 1 colem colem 336 Feb 4 17:33 calibrate_kiosk.py
-rw-rw-r-- 1 colem colem 6672 Feb 4 18:27 kiosk_calibration.log
-rw-r--r-- 1 colem colem 11 Feb 4 18:06 kiosk_db.txt
-rw-rw-r-- 1 colem colem 47 Feb 4 17:58 kiosk_scanner.log
-rw----- 1 colem colem 0 Feb 4 17:51 nohup.out
```

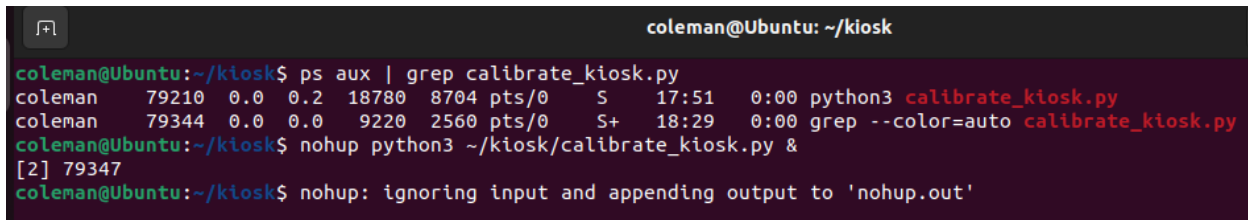
Process Verification

```
ps aux | grep calibrate_kiosk.py
```

No active process confirmed the simulated service failure so I restarted the script.

```
nohup python3 ~/kiosk/calibrate_kiosk.py &
```

Figure 10: Process command and restart

A terminal window titled 'coleman@Ubuntu: ~/kiosk' showing the following commands and output:

```
coleman@Ubuntu:~/kiosk$ ps aux | grep calibrate_kiosk.py
coleman  79210  0.0  0.2 18780  8704 pts/0    S   17:51   0:00 python3 calibrate_kiosk.py
coleman  79344  0.0  0.0   9220  2560 pts/0    S+  18:29   0:00 grep --color=auto calibrate_kiosk.py
coleman@Ubuntu:~/kiosk$ nohup python3 ~/kiosk/calibrate_kiosk.py &
[2] 79347
coleman@Ubuntu:~/kiosk$ nohup: ignoring input and appending output to 'nohup.out'
```

Network Diagnostics

Connectivity testing included:

```
ping 8.8.8.8
```

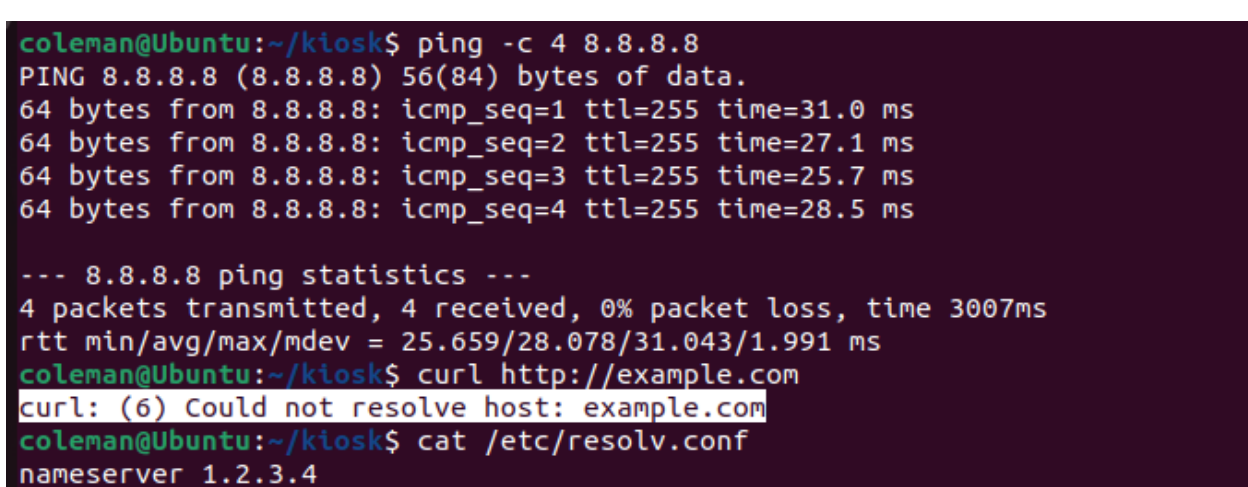
```
curl http://example.com
```

```
cat /etc/resolv.conf
```

Observations:

- Ping worked (network reachable)
- Curl failed (DNS or firewall issue)
- DNS server misconfigured

Figure 11: Network check results

A terminal window titled 'coleman@Ubuntu: ~/kiosk' showing the following commands and output:

```
coleman@Ubuntu:~/kiosk$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=31.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=27.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=25.7 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=28.5 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 25.659/28.078/31.043/1.991 ms
coleman@Ubuntu:~/kiosk$ curl http://example.com
curl: (6) Could not resolve host: example.com
coleman@Ubuntu:~/kiosk$ cat /etc/resolv.conf
nameserver 1.2.3.4
```


System Health Checks

Basic resource checks ensured the system itself wasn't the cause:

```
df -h
```

```
free -h
```

```
top
```

These checks confirmed the kiosk failures were configuration-based rather than resource exhaustion.

Scripted Fixes & Automation

To simulate real support workflows, I created Bash scripts to automate recurring fixes.

Calibration Restart Script

```
chmod +x ~/kiosk/calibrate_kiosk.py  
pkill -f calibrate_kiosk.py  
nohup python3 ~/kiosk/calibrate_kiosk.py &  
echo "Calibration script restarted and permissions corrected"
```

The purpose of restarting the script is to reduce manual recovery time and standardize service restarts.

Scanner/Cutter Reset Script

```
echo "Cutter cleared" > ~/kiosk/kiosk_scanner.log  
echo "Scanner reset" >> ~/kiosk/kiosk_scanner.log
```

This simulated clearing a physical jam.

Network Recovery Script

```
sudo iptables -F  
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

This restored connectivity by removing firewall blocks and correcting DNS.

Figure 12,13,14: Restart, Reset, and Recovery results

```
colem@Ubuntu: ~/kiosk

colem@Ubuntu:~/kiosk$ chmod +x calibrate_kiosk.py
colem@Ubuntu:~/kiosk$ pkill -f calibrate_kiosk.py
colem@Ubuntu:~/kiosk$ nohup python3 calibrate_kiosk.py &
[3] 81299
[1] Terminated          nohup python3 calibrate_kiosk.py
[2] Terminated          nohup python3 ~/kiosk/calibrate_kiosk.py
colem@Ubuntu:~/kiosk$ nohup: ignoring input and appending output to 'nohup.out'

colem@Ubuntu:~/kiosk$ echo "Calibration script restarted and permissions corrected"
Calibration script restarted and permissions corrected
colem@Ubuntu:~/kiosk$

colem@Ubuntu:~/kiosk$ echo "Cutter cleared" > ~/kiosk/kiosk_scanner.log
colem@Ubuntu:~/kiosk$ echo "Scanner reset" >> ~/kiosk/kiosk_scanner.log
colem@Ubuntu:~/kiosk$ echo "Scanner and cutter logs cleared"
Scanner and cutter logs cleared

colem@Ubuntu:~/kiosk$ sudo iptables -F
colem@Ubuntu:~/kiosk$ echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
nameserver 8.8.8.8
colem@Ubuntu:~/kiosk$ echo "Network issues cleared"
Network issues cleared
```

Documentation & Internal Knowledge Base Creation

To mirror real support environments, I documented solutions in a README-style internal knowledge base.

Example topics included:

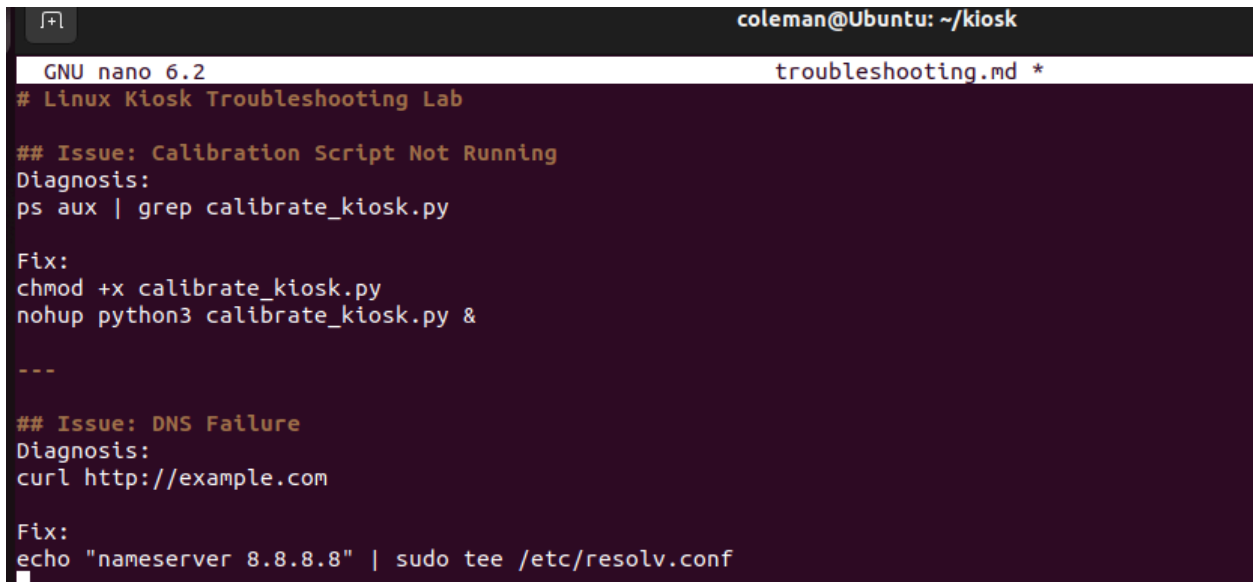
- Calibration failures
- Scanner jams
- File permission errors

Each entry contained:

- Symptoms
- Diagnostic commands
- Fix steps
- Lessons learned

This step demonstrates not just technical ability, but operational maturity and documenting solutions for future technicians.

Figure 15: README.md content

A screenshot of a terminal window with a dark background. The title bar at the top shows 'coleman@Ubuntu: ~/kiosk'. The terminal content shows the nano 6.2 editor editing a file named 'troubleshooting.md'. The file content includes a title '# Linux Kiosk Troubleshooting Lab', followed by a section '## Issue: Calibration Script Not Running' with 'Diagnosis:' and 'ps aux | grep calibrate_kiosk.py', and a 'Fix:' section with 'chmod +x calibrate_kiosk.py' and 'nohup python3 calibrate_kiosk.py &'. After three dashes, another section '## Issue: DNS Failure' is shown with 'Diagnosis:' and 'curl http://example.com', and a 'Fix:' section with 'echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf'.

```
GNU nano 6.2 troubleshooting.md *
# Linux Kiosk Troubleshooting Lab

## Issue: Calibration Script Not Running
Diagnosis:
ps aux | grep calibrate_kiosk.py

Fix:
chmod +x calibrate_kiosk.py
nohup python3 calibrate_kiosk.py &

---

## Issue: DNS Failure
Diagnosis:
curl http://example.com

Fix:
echo "nameserver 8.8.8.8" | sudo tee /etc/resolv.conf
```

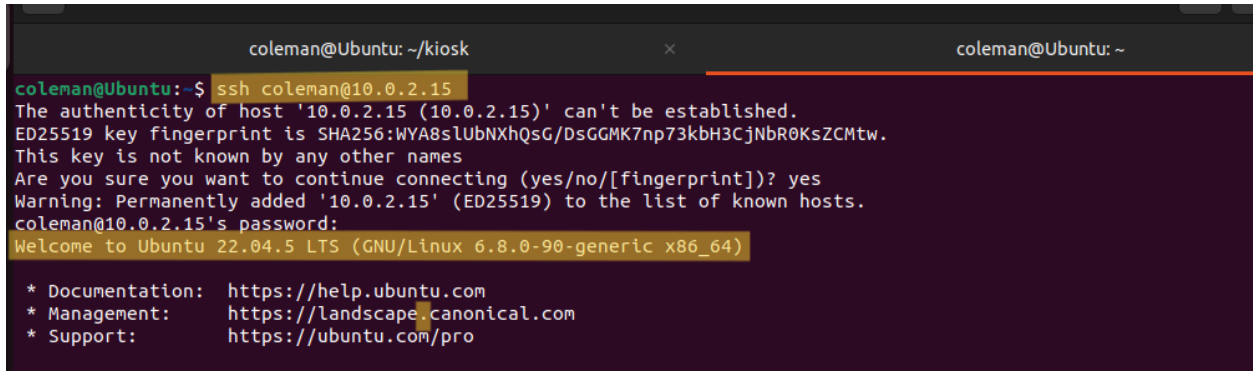
Remote Support Simulation (SSH)

After confirming local fixes worked, I simulated remote troubleshooting using SSH.

ssh coleman@10.0.2.15

From the remote session I was able to navigate to the kiosk directory, verify logs, check services, and confirm network fixes. This simulated real world remote incident response without physical access.

Figure 16: SSH login

A screenshot of a terminal window showing an SSH login process. The terminal title bar shows 'coleman@Ubuntu: ~/kiosk'. The user enters 'ssh colem@10.0.2.15'. The terminal output shows a warning about the host's authenticity, a fingerprint, and a confirmation to add the host to the known hosts list. The user enters 'yes'. The terminal then shows the Ubuntu 22.04.5 LTS login banner and a list of links for documentation, management, and support.

```
coleman@Ubuntu: ~/kiosk
coleman@Ubuntu:~$ ssh colem@10.0.2.15
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.
ED25519 key fingerprint is SHA256:WYA8slUbNXhQsG/DsGGMK7np73kbH3CjNbR0KsZCMTw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.2.15' (ED25519) to the list of known hosts.
coleman@10.0.2.15's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-90-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
```

Optional Enhancements Not Implemented

Additional/optional improvements included:

Cron Job Automation

`crontab -e`

```
*/30 * * * * nohup python3 ~/kiosk/calibrate_kiosk.py &
```

This would cause the calibration to restart automatically (**every 30 mins, any hour, any day, any month, any day of the week**).

Conclusion

This project demonstrated a complete, hands-on approach to troubleshooting a Linux-based kiosk environment, from simulating hardware and software failures to diagnosing issues with logs, permissions, services, and network connectivity. By creating and resolving common errors—such as calibration failures, scanner jams, file permission issues, and connectivity problems—I practiced structured diagnostics, methodical problem-solving, and clear documentation of each step. Remote administration via SSH reinforced real-world support workflows, allowing testing and fixes without physical access. For process management, I used `nohup` instead of `systemd` to quickly restart the calibration script during testing, reflecting field-style troubleshooting where immediate recovery takes priority over long-term deployment. Overall, this project highlights practical Linux skills, troubleshooting methodology, automation with Bash scripts, and operational thinking—key abilities directly applicable to technical support roles in environments like KeyMe’s kiosks.

